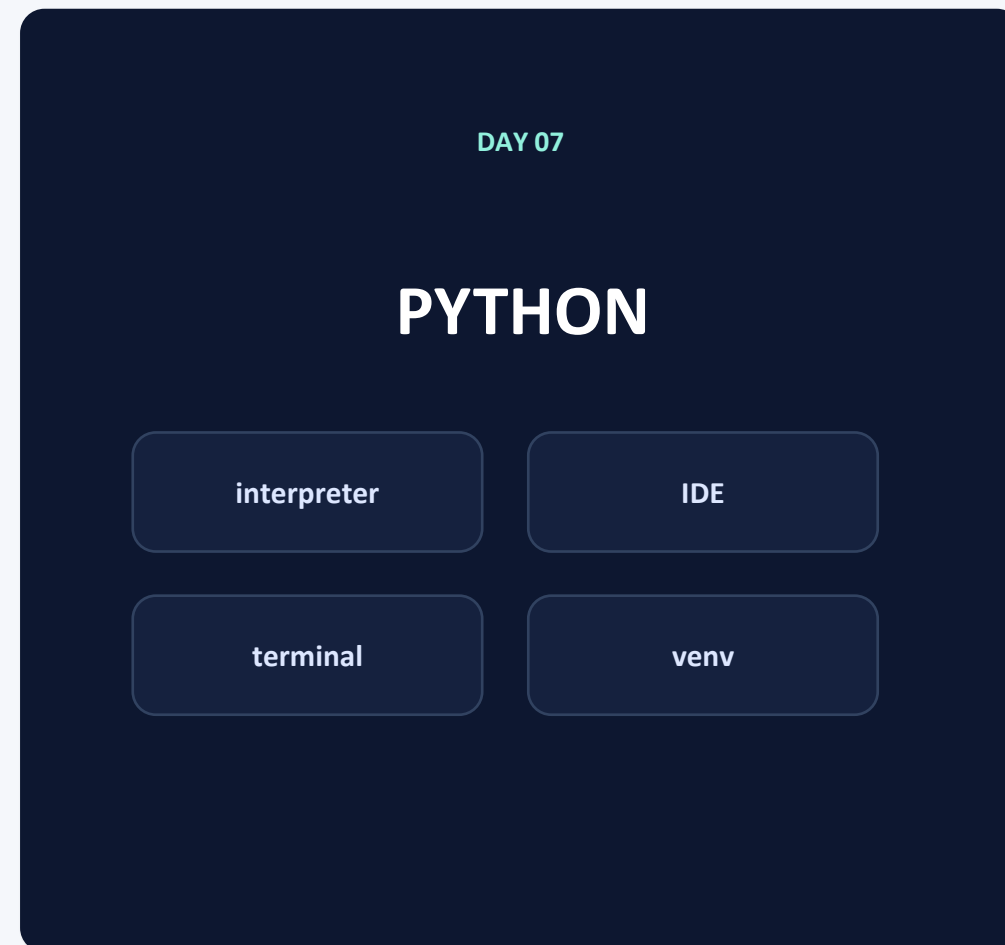


Python и рабочее окружение

Запустить первый Python-скрипт и понять роли интерпретатора, IDE, терминала, виртуального окружения и pip.



ЦЕЛЬ ДНЯ

Что ты сможешь после этого дня

Конкретные действия вместо списка терминов.

- 01 Запустить первый Python-скрипт и понять роли интерпретатора, IDE, терминала, виртуального окружения и pip.
- 02 Объяснить минимум 5 терминов темы своими словами.
- 03 Прочитать показанный пример кода и предсказать результат.
- 04 Самостоятельно решить похожую задачу после разбора.

СВЯЗЬ

Новая тема не появляется из ниоткуда

Связываем сегодняшний шаг с уже построенной картой курса.

ВЧЕРА

Алгоритмы и структуры данных

Что уже есть в нашей модели.



СЕГОДНЯ

Python и рабочее окружение

Вчера: Алгоритмы и структуры данных. Сегодня мы добавляем следующий слой: python и рабочее окружение.

ЗАЧЕМ

Зачем мне это?

Каждая тема решает конкретную проблему.

ПРОБЛЕМА → ПОЛЬЗА

Надёжная рабочая среда убирает половину новичковых проблем: «почему команда не работает», «куда установился пакет», «какой Python запускается».

Вопрос себе

Какую реальную проблему решает сегодняшняя тема?

Ответь до следующего слайда.

Сначала — человеческий образ

Потом обязательно связываем его с настоящей техникой.

БЫТОВОЙ ОБРАЗ

Python-интерпретатор — переводчик инструкций, IDE — рабочий стол программиста, терминал — прямой пульт управления, venv — отдельная коробка инструментов проекта.

ПЕРЕВОД В IT

Python

язык программирования, ориентированный на читаемый код

interpreter

программа, которая выполняет код Python

IDE

редактор с инструментами запуска, подсказок и отладки

Слой 1: Python • interpreter • IDE

Сначала смысл, затем профессиональный термин.

**Python**

язык программирования, ориентированный на читаемый код

**interpreter**

программа, которая выполняет код Python

**IDE**

редактор с инструментами запуска, подсказок и отладки

Слой 2: terminal • venv • pip

Сначала смысл, затем профессиональный термин.

**terminal**

текстовый интерфейс для выполнения команд

**venv**

изолированное окружение Python для конкретного проекта

**pip**

инструмент установки Python-пакетов

Слой 3: script • REPL

Сначала смысл, затем профессиональный термин.

**script**

файл с командами программы

**REPL**

интерактивная среда выполнения команд

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Слой 4: library • package manager

Сначала смысл, затем профессиональный термин.

library

готовый переиспользуемый программный код

package manager

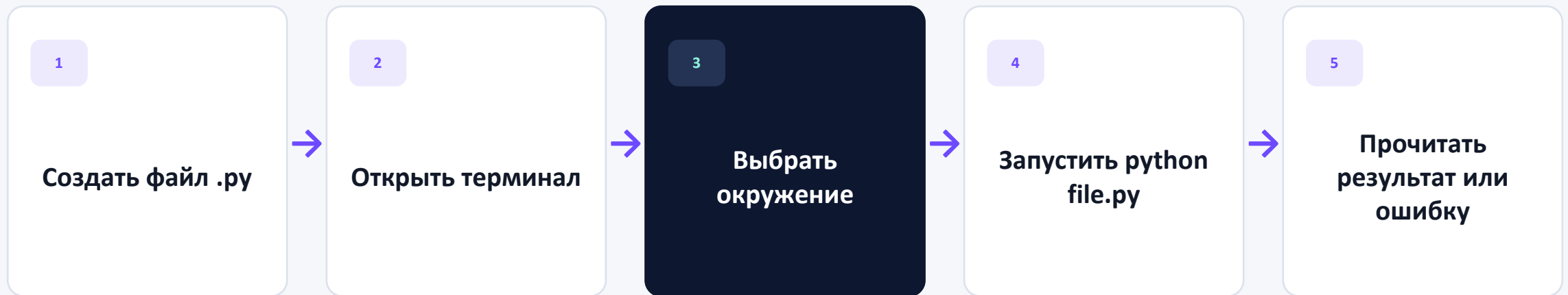
инструмент установки и обновления зависимостей

Не заучивай формулировку. Придумай свой пример использования каждого термина.

СХЕМА

Как это работает по шагам

Проследи движение слева направо.



Проговори эту цепочку своими словами, не подсматривая в текст.

Первый минимальный пример

Смотрим только на одну основную идею.



day07_step1

```
print("Привет, Python!")
```

ЧТО ИСКАТЬ

- Где данные?
- Где действие?
- Где результат?
- Что изменится, если заменить одно значение?

Разбираем пример построчно

Код не должен оставаться магической конструкцией.

```
print("Привет, Python!")
```

Показываем результат, чтобы увидеть состояние программы.

РЕЗУЛЬТАТ

Код → выполнение → результат

Всегда связываем запись с тем, что реально получилось.



до запуска

```
print("Привет, Python!")
```

РЕЗУЛЬТАТ

Привет, Python!

ИЗМЕНИЛИ

Теперь изменим пример

Сначала предскажи результат. Не запускай код сразу.



changed

```
name = "Анна"  
print("Привет,", name)
```



**Что появится в
результате?**

Сформулируй ответ словами.

ОТВЕТ

Сверяем прогноз

Не просто смотри ответ — объясни, почему он именно такой.

РЕЗУЛЬТАТ

Привет, Анна

ПОЧЕМУ

Python/система выполняет шаги в записанном порядке. Изменённые данные или действие дают новый результат.

Сравни с первым примером: что изменилось?

ОШИБКА

Как писать неправильно

Ошибка — отдельный учебный материал, а не повод её скрывать.



wrong

```
print(Привет)
```

СТОП

**Что здесь может сломаться
или ввести нас в
заблуждение?**

Сначала найди проблему сам.

Почему возникает проблема

Переводим техническую причину на обычный язык.

Привет — текст, поэтому нужны кавычки. Без них Python ищет переменную с таким именем.

Алгоритм отладки: симптом → конкретная строка/шаг → причина → минимальное исправление → повторная проверка.

ИСПРАВЛЕНИЕ

Исправленный вариант

Меняем только то, что действительно было причиной.



fixed

```
print("Привет")
```

ПРАВИЛО

Не переписывай всё решение. Найди минимальное осмысленное исправление.

Что изменилось после исправления?

Ответь без запуска.

- 1 Исчезла ли исходная причина проблемы?
- 2 Какой результат/поведение теперь ожидаем?
- 3 Как проверить это одним маленьким примером?

Сначала увидим полностью разобранный пример

Только после него будет самостоятельная практика.

ЗАДАЧА

Создать hello.py, сохранить имя в переменную и вывести персональное приветствие.

Не начинаем с кода. Сначала переводим задачу на человеческий язык.

КАК ДУМАТЬ

Как рассуждает программист

Разбиваем большую формулировку на маленькие действия.

- 1 Создать файл `hello.py`.
- 2 Записать имя как строку.
- 3 Передать текст и переменную в `print()`.
- 4 Запустить файл через Python.
- 5 Сравнить вывод с ожиданием.

Шаг 1: собираем основу

Не показываем 20 строк сразу — строим решение частями.



solution_part1

```
name = "Alex"  
print("Привет,", name)
```

Проверь

- Какие данные уже есть?
- Какой шаг плана реализован?
- Что ещё осталось добавить?

ИТОГОВЫЙ КОД

Шаг 2: итоговое решение

Теперь видим программу/запрос целиком.



solution_final

```
name = "Alex"  
print("Привет,", name)  
print("Python запущен успешно")
```

Проверяем результат

Сравниваем то, что ожидали, с тем, что получили.

РЕЗУЛЬТАТ

Привет, Alex
Python запущен успешно

ГЛАВНОЕ

Задача была решена не потому, что мы «знали готовый код», а потому что сначала построили понятный план.

ПОПРОБУЙ САМ

Практика 1

Задача очень похожа на разобрannую — но теперь первый ход за тобой.

Создай `about_me.py`: сохрани имя и город в переменные и выведи две понятные строки.

Сначала выпиши 2–5 шагов решения словами.

ПОДСКАЗКА

Подсказка — только направление

Попробуй ещё раз до просмотра готового решения.

Каждый текст бери в кавычки, а данные храни под понятными именами.

Вопрос себе: какие конструкции сегодняшнего дня нужны, а какие пока лишние?

Один из правильных вариантов

Сравни логику, а не количество строк.



practice_solution

```
name = "Mia"  
city = "Tallinn"  
print("Меня зовут", name)  
print("Я живу в", city)
```

Ожидаемый результат

Если твой код устроен иначе, но даёт правильное и понятное решение — это нормально.

РЕЗУЛЬТАТ

Меня зовут Mia
Я живу в Tallinn

Сравни

- Результат
- Понятность имён
- Лишние шаги
- Обработку ошибок

Практика 2 — мини-проект

Соединяем несколько элементов дня в одну маленькую закончённую задачу.

ЗАДАНИЕ

Сделай мини-визитку из трёх переменных: name, goal, days.

Попробуй сначала без подсказки. Разрешено подсматривать в сегодняшние примеры.

Разбор мини-проекта

Вот один рабочий путь.



mini_project

```
name = "Mia"  
goal = "выучить Python"  
days = 30  
print(name)  
print(goal)  
print("План:", days, "дней")
```

РЕЗУЛЬТАТ ПРОЕКТА

Что должно получиться

Связываем итог с задачей и проверяем смысл.

РЕЗУЛЬТАТ

Mia
выучить Python
План: 30 дней

Объясни каждую важную строку/команду своими словами. Если не можешь — вернись к конкретному месту, а не перечитывай весь день.

Четыре ловушки новичка

Лучше увидеть их здесь, чем случайно закрепить как привычку.

01 **Запускать не тот файл.**

02 **Забывать сохранить изменения перед запуском.**

03 **Путать терминал и Python-консоль.**

04 **Устанавливать зависимости вне нужного окружения.**

Проверь себя: 1–2

Не листай дальше, пока не сформулируешь ответ.

1

Что такое «Python» и какую проблему это понятие помогает описать?

2

Чем «Python» отличается от «terminal»? Объясни без копирования определения.

Проверь себя: 3–4

Здесь проверяем понимание кода и процесса мышления.

- 3 Какой результат даёт первый минимальный пример и почему?
- 4 Какую ошибку/риск мы разобрали сегодня и какое минимальное исправление нужно?

ОТВЕТЫ

Ответы к вопросам 1–2

Формулировка может отличаться — важен смысл.

1 Python — язык программирования, ориентированный на читаемый код.

2 Python: язык программирования, ориентированный на читаемый код. terminal: текстовый интерфейс для выполнения команд.

ОТВЕТЫ

Ответы к вопросам 3–4

Сверяем причинно-следственную связь.

3 **Привет, Python!**

4 **Привет — текст, поэтому нужны кавычки. Без них Python ищет переменную с таким именем.**

ИТОГ

Сегодня мы научились

Не перечисляем слова — фиксируем новые способности.

- ✓ Запустить первый Python-скрипт и понять роли интерпретатора, IDE, терминала, виртуального окружения и pip.
- ✓ Читать и объяснять пример темы «Python и рабочее окружение».
- ✓ Находить типичную ошибку и формулировать причину.
- ✓ Решать похожую задачу по шагам, а не угадывать готовый код.

ДАЛЬШЕ

Что будет завтра

Связываем сегодняшний результат со следующим шагом.

Переменные и типы данных

Завтра: Переменные и типы данных. Сегодняшняя модель станет опорой для следующего шага.

Сегодняшняя тема не заканчивается здесь — она станет инструментом в следующем дне.